

ChipChat Assignment Report

Student: Abdulhaseeb Khan

Model: OpenAI gpt-4o

1. Completed Notebook Examples

All eight examples were completed. Only one passed on the first turn; six of the eight first-turn failures involved module or port names that the prompt did not specify.

#	Example	Turns	First-turn failure
1	binary_to_bcd	1	none
2	sequence_detector	2	registered output; testbench needs Mealy
3	shift_register	5	provided testbench unsatisfiable; design correct
4	abro_state_machine	3	port names, nonstandard semantics, delta-cycle race
5	dice_roller	2	port name mismatch
6	lfsr	2	syntax error, names, wrong taps
7	sequence_generator	2	port name, off-by-one timing
8	traffic_light	2	SystemVerilog syntax, module name, counter timing

Example 1: binary_to_bcd (1 turn)

GPT-4o produced a double-dabble (shift-and-add-3) converter with the module name `binary_to_bcd_converter` and ports `binary_input` / `bcd_output`. This was the only prompt where I explicitly named the module, and accordingly it was the only example whose ports matched the testbench on the first try. Simulation output: *All test cases passed!* No feedback was needed.

Example 2: sequence_detector (2 turns)

The first design used a registered `sequence_found` output (Moore-style), and on closer inspection it lacked states for the repeated 110 and for 011, so the FSM could never reach the end of the 8-symbol sequence. Simulation failed at cycle 8: *Expected: 1, Got: 0*. I fed back that the testbench samples the output on the same clock edge the eighth symbol is applied, before a registered output would update, so it must be combinational (Mealy). I also pointed out the missing states and required plain Verilog-2001. The corrected design used a 3-bit match counter with a combinational assign and passed all checks.

Example 3: shift_register (5 turns)

The design was a correct SIPO shift register from turn 1; I only renamed the reset port from `n_reset` to `reset_n` in turn 2 to match the testbench. Both versions failed the provided GUIDE testbench at test case 0: it expects `data_out = 00001010` before reset has ever been asserted and while `shift_enable = 0`, which no causal implementation can produce. I ran a sweep of 32 shift-register variants and confirmed every one fails. The only design that passes hardcodes an initial value and right-shifts unconditionally, ignoring both inputs.

In turns 3 to 5 I tested whether the model could identify and repair the defective testbench. In turn 3, GPT-4o correctly concluded the testbench was at fault, but its replacement had a sampling race and printed "All tests passed!" after failure messages. In turn 4, it fixed the race but declared a reg inside an unnamed block (SystemVerilog only) and still had an extra clock edge per test. In turn 5, with all defects spelled out, it finally produced a sound Verilog-2001 self-checking testbench that passed.

Example 4: abro_state_machine (3 turns)

Turn 1 produced a textbook ABRO with four states, module named ABRO, and reset port `nreset`, none of which matched the testbench. In turn 2, I fixed the names and re-specified the testbench's nonstandard machine: B alone returns to idle 0001; A alone goes to 0010; both go to 0100 with `O = A & B` combinational. The design still failed at test case 2 due to a delta-cycle race: the testbench changes inputs and checks State in the same timestep as the

clock edge. In turn 3, I asked for a single clocked block computing state directly from {A,B}, and all test cases passed.

Example 5: dice_roller (2 turns)

The turn-1 design was functionally correct: it used `{$random} % max_number + 1` with the right die-select mapping (4/6/8/20-sided). The only defect was the reset port name (reset_n vs the testbench's rst_n). I fed back the testbench's instantiation, and in turn 2 the model renamed it. The testbench ran 1000 rolls per die type and checked range. Output: *Testbench completed successfully.*

Example 6: lfsr (2 turns)

Turn 1 had three defects: a wire declared inside the always block (syntax error), module LFSR with port rst_n instead of lowercase lfsr with reset_n, and feedback taps `[7]^[6]^[3]^[1]` where the testbench reference uses `[0]^[3]^[5]^[6]`. The taps are pure convention and not derivable from the prompt, so I had to supply the exact polynomial. I fed back all three issues, and the turn-2 design fixed them. *Simulation successful. All test cases passed.*

Example 7: sequence_generator (2 turns)

Turn 1 named the reset nreset (testbench expects reset_n) and had an off-by-one: the clocked case re-output the current-state value while incrementing state, so data lagged the checker by one cycle. I fed back the port rename and explained that the case needed to emit the value for state+1, wrapping from 8'h8D back to 8'hAF. All three test cases passed: sequence generation, repetition, and disable-hold.

Example 8: traffic_light (2 turns)

Turn 1 used SystemVerilog typedef enum logic (rejected by iverilog), named the module traffic_light instead of traffic_light_fsm, and used a down-counter with suspect boundary timing. I restated the exact timing contract (GREEN after the 32nd enabled edge, YELLOW after 20 more, RED after 7 more) and asked for a single clocked block with an up-counter starting at 1. All four test cases passed at the exact 32/20/7 cycle boundaries.

2. CPU Example: 8-bit Accumulator Processor (4 turns)

Starting prompt (From the assignment PDF):

"Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?"

Turn 1. The model replied with a design plan (requirements, accumulator architecture, minimal ISA, datapath, control, memory, verification) but no decisions or code.

Turn 2. I asked it to fix the ISA and produce cpu.v and cpu_tb.v. It proposed 8-bit instructions with a 4-bit opcode and 4-bit immediate/address:

Opcode	Mnemonic	Semantics
0000	LOAD imm	acc = imm
0001	ADD imm	acc += imm
0010	SUB imm	acc -= imm
0011	AND imm	acc &= imm
0100	OR imm	acc = imm
0101	XOR imm	acc ^= imm
0110	STORE addr	mem[addr] = acc
0111	JUMP addr	pc = addr

The cpu.v had a 4-bit pc, 8-bit acc, a 16x8 internal unified memory, and single-cycle execution. Its testbench ended in \$stop, which hangs Icarus vvp, so I had to interrupt the cell.

Turn 3. I told it to use \$finish, make the testbench self-checking, and halt with a jump-to-self. The new testbench had three bugs, which I confirmed by simulation (acc=3, pc=3): the halt jump was encoded 01111111 (address 15, not 7 as commented); STORE targeted mem[1], overwriting the ADD instruction; the pc then fetched uninitialized memory, wrapped to 0, and re-executed the corrupted program.

Turn 4. I named all three bugs and specified the program (LOAD 5; ADD 5; SUB 3; XOR 1; STORE mem[10]; JUMP 5) with expected acc = 6, mem[10] = 6, pc = 5.

Final result:

PASS: Accumulator value is correct.

PASS: Memory value at address 10 is correct.

PASS: Program counter is correctly parked at halt.

Architectural choices: fixed 8-bit instructions (4-bit opcode + 4-bit operand), acc and pc as the only registers, single-cycle fetch/decode/execute, 16-byte unified memory inside the CPU that STORE can write (so code can self-modify), immediate-only operands.

Limitations I noted but did not fix: the 128 flip-flops of internal memory work against the 1000-cell budget (the paper kept memory external); the data_bus inout is declared but never driven; memory_address is 8 bits driven by a 4-bit pc; no conditional branch, so no data-dependent control flow. The CPU design was correct at its first code turn; all my debugging effort went into the testbench.

3. How Conversation and Human Feedback Influenced the Design Process

Small examples. My role was mostly to relay constraints the model could not see: exact module and port names, output timing conventions (Mealy vs Moore), LFSR taps, and the simulator's dialect. Every design failure was fixed in one turn once I quoted the testbench's instantiation and named the exact defect. Where my feedback was vague, the model repeated the same bug.

CPU and testbench authoring. With no testbench provided, the model had to write its own, and this is where I spent the most effort. It produced racy, mis-encoded, or unconditionally-passing testbenches four times (Example 3 turns 3 and 4; CPU turns 2 and 3), and twice misread its own vectors. My job shifted from supplying the spec to acting as the verification oracle: running the code, reading the failure, and diagnosing it. That turned out to be the less automatable half of the loop, and it mattered more as the design grew.

Is feedback more valuable for complex designs? In my experience, yes. On the small examples, most feedback could have been avoided by a better initial prompt. On the CPU, my feedback caught errors that no prompt would have prevented, because they were in code the model wrote and then misdescribed.

Strengths I observed. Fast convergence on concrete feedback; the transcript serves as complete design provenance; first attempts were structurally right on every well-known idiom (double-dabble, LFSRs, one-hot FSMs, accumulator ISA).

Limitations I observed. Prose underdetermines interfaces and timing; the model is a weak testbench author and a weak diagnostician of its own output; SystemVerilog constructs appear unprompted; regeneration is nondeterministic, so only the pasted, verified designs are reproducible.

4. Failures, Debugging Steps, and Improvements Across Iterations

- **Example 3:** I proved the provided GUIDE testbench is defective (unsatisfiable vectors) by running a 32-variant sweep and showing the only passing "design" ignores all inputs. I kept the original file alongside the model-authored replacement for reproducibility.
- **Examples 2 and 4:** The provided testbenches check outputs in the same timestep as the clock edge, before nonblocking updates land, which forced me to ask for Mealy outputs and single-block FSMs.
- **Examples 6 and 8:** SystemVerilog constructs (typedef enum, declarations in unnamed blocks) were rejected by iverilog; I fixed these by explicitly requiring Verilog-2001 in my feedback.

- **CPU:** \$stop hung vvp; a mis-commented jump encoding plus a STORE into the program region produced accidental self-modifying code. I fixed it by specifying the exact program and expected state.
- **Process improvement:** I found that including the testbench's instantiation block in the feedback eliminated every naming failure in one round. In hindsight this should have been in the initial prompt for all examples.